

The Black Ninjas and the Sniper: On Robustness of Population Protocols

Benno Lossin, Philipp Czerner ,
Javier Esparza , Roland Guttenberg , Tobias Prehn

{lossin, czerner, esparza, guttenbe}@in.tum.de
Department of Informatics, TU München, Germany

Abstract. Population protocols are a model of distributed computation in which an arbitrary number of indistinguishable finite-state agents interact in pairs to decide some property of their initial configuration. We investigate the behaviour of population protocols under adversarial faults that cause agents to silently crash and no longer interact with other agents. As a starting point, we consider the property “the number of agents exceeds a given threshold t ”, represented by the predicate $x \geq t$, and show that the standard protocol for $x \geq t$ is very fragile: one single crash in a computation with $x := 2t - 1$ agents can already cause the protocol to answer incorrectly that $x \geq t$ does not hold. However, a slightly less known protocol is *robust*: for any number $t' \geq t$ of agents, at least $t' - t + 1$ crashes must occur for the protocol to answer that the property does not hold.

We formally define robustness for arbitrary population protocols, and investigate the question whether every predicate computable by population protocols has a robust protocol. Angluin et al. proved in 2007 that population protocols decide exactly the Presburger predicates, which can be represented as Boolean combinations of threshold predicates of the form $\sum_{i=1}^n a_i \cdot x_i \geq t$ for $a_1, \dots, a_n, t \in \mathbb{Z}$ and modulo predicates of the form $\sum_{i=1}^n a_i \cdot x_i \bmod m \geq t$ for $a_1, \dots, a_n, m, t \in \mathbb{N}$. We design robust protocols for all threshold and modulo predicates. We also show that, unfortunately, the techniques in the literature that construct a protocol for a Boolean combination of predicates given protocols for the conjuncts do not preserve robustness. So the question remains open.

For Joost-Pieter, Honorary Sensei of the Black Ninjas.

1 Introduction

The Black Ninjas were used in [5] to provide a gentle introduction to population protocols, a model of distributed computation introduced by Angluin et al. in [3] and since then very much studied. We quote (with some changes) the first lines of the introduction to [5]:

The Black Ninjas are an ancient secret society of warriors, so secret that its members do not even know each other or how many they are. When there is a matter to discuss, Sensei, the founder of the society, sends individual messages to each ninja, asking them to meet.

As it happens, all ninjas have just received a note asking them to meet in a certain Zen garden at midnight, wearing their black uniform, in order to attack a fortress of the Dark Powers. When the ninjas reach the garden in the gloomy night, the weather is so dreadful that it is impossible to see or hear anything at all. This causes a problem, because the ninjas should only attack if they are at least 64, and there are always no-shows: Some ninjas are wounded, others are under evil spells, and others still have tax forms to fill.

Is there a way for the ninjas to find out, despite the adverse conditions, if at least 64 of them have shown up?

A first protocol. Yes, there is a way. Sensei has looked at the weather forecast, and the note sent to the ninjas contains detailed instructions on how to proceed after they reach the garden. Each ninja must bring pebbles in their pockets, and a pouch initially containing one pebble. When they reach the garden, they must start to wander *randomly* around the garden. When two ninjas with n_1 and n_2 pebbles in their pouches happen to bump into each other, they compute $n_1 + n_2$ and proceed as follows:

- If $n_1 + n_2 < 64$, then one ninja gives her pebbles to the other, that is, after the encounter their pouches contain $n_1 + n_2$ and 0 pebbles, respectively.
- If $n_1 + n_2 \geq 64$, both ninjas put 64 pebbles in their pouches.

Formally, at every moment in time each ninja is in one state from the set $Q := \{0, 1, \dots, 64\}$, representing the number of pebbles in her pouch. The possible transitions are

$$n_1, n_2 \mapsto \begin{cases} n_1 + n_2, 0 & \text{if } n_1 + n_2 < 64 \\ 64, 64 & \text{otherwise} \end{cases}$$

The protocol can be visualized as a Petri net with one place for each state, and one Petri net transition for each transition of the protocol. The Petri net representation of the protocol for attack threshold 4 instead of 64 is shown on the left of [Figure 1](#).

Sensei reasons as follows: If at least 64 ninjas show up, then eventually two ninjas with at least 64 pebbles between them interact. These ninjas move to state 64, and from then on word quickly spreads, and eventually all ninjas are in state 64, and stay there forever. If, on the contrary, less than 64 ninjas show up, then no ninja ever reaches state 64. In

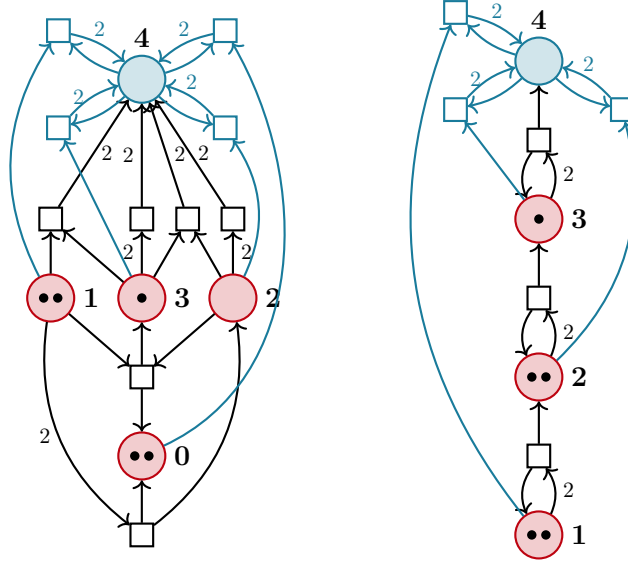


Figure 1: Petri nets for the first (left) and second (right) protocols with attack threshold 4. Accepting states are shaded in blue, rejecting states in red. The number of pebbles (left) or the level (right) of a state are written next to its corresponding place in boldface. The tokens in the places show configurations reachable from the initial configuration with 5 ninjas. The accumulation transitions are colored in blue.

both cases, the ninjas eventually reach a *consensus*: all of them are in state 64, or none is, and that consensus does not change, it is *stable*. Further, since Sensei knows that the society consists of 200 ninjas—a fact the ninjas themselves ignore—and she is good at math, she computes that after 10 minutes the consensus has been reached with high probability (ninjas move very fast!). So her final instruction to the ninjas is: Execute the protocol for 10 minutes, and then attack if you are in state 64.

1.1 The sniper and a second protocol.

The ninjas use the protocol to successfully attack several fortresses of the Evil Powers. However, a spy finds out the point of their next gathering, and the Evil Powers send a sniper equipped with a night vision device and a powerful gun whose bullets can silently vaporize a ninja leaving no trace. No less than 126 ninjas show up at the gathering, but the sniper manages to prevent the attack by eliminating one single ninja! The sniper lets the ninjas wander around, executing the protocol, until they reach a configuration in which two ninjas have pouches with 63 pebbles each, and all other pouches are empty. Right before these two ninjas bump into each other, the sniper eliminates one of them. The other ninjas, unaware of this, are left in a configuration in which one ninja has 63 pebbles in her pouch, and the pouches of the other 124 ninjas are empty. The ninjas keep interacting, but none ever reach state 64.

When Sensei finds out about the sniper, she initially despairs. But then she realizes:

munitions are expensive, and the sniper can carry only a few rounds, so perhaps there is a remedy. So she searches for a *robust* protocol. If $n \geq 64$ ninjas reach the garden, the sniper can only prevent an attack by eliminating more than $n - 64$ ninjas. Clearly, this is the best she can achieve, because if the sniper succeeds in removing more than $n - 64$ ninjas, then the ninjas should not attack anyway.

Sensei is a capable leader, and she finds a robust protocol! The states are now the numbers 1 to 64. Sensei visualizes these numbers as the floors or levels of a tower with 64 levels, and visualizes a ninja in state ℓ as occupying the ℓ -th level. Initially all ninjas are in the first level. When two ninjas bump into each other, they interact as follows:

- if they are both in the same level, say the ℓ -th, and $\ell < 64$, then one of them moves to level $\ell + 1$, and the other stays at level ℓ .
- if one of them is in level 64, then the other moves to level 64 too (if not yet there);

Formally, at every moment in time each ninja is in a state from the set $Q := \{1, 2, \dots, 64\}$. The possible transitions are

$$\ell_1, \ell_2 \mapsto \begin{cases} \ell_1 + 1, \ell_2 & \text{if } \ell_1 = \ell_2, \text{ and } \ell_1 < 64 \\ 64, 64 & \text{if } \ell_1 = 64 \text{ or } \ell_2 = 64 \\ \ell_1, \ell_2 & \text{otherwise} \end{cases}$$

Again, Sensei asks the ninjas to execute the protocol for a certain time, and then attack if they are in level 64. The protocol can again be visualized as a Petri net, shown on the right of [Figure 1](#).

Sensei first shows that the protocol works correctly in the absence of the sniper. Eventually, each level is populated by at most one ninja; indeed, any two ninjas in the same level eventually interact and one of them moves up. Therefore, if there are at least 64 ninjas, then one of them eventually reaches the top of the tower, and eventually brings all other ninjas to the top.

To prove that the protocol is robust, Sensei observes that the argument above works not only for the initial configuration with all ninjas in the ground level, but *for any configuration with at least 64 ninjas*. Indeed, in any such configuration either one ninja is already at the top, or, by the pigeonhole principle, eventually one ninja moves one level up. Therefore, unless the sniper brings the number of ninjas below 64, the ninjas attack.

Sensei's question. Sensei is not satisfied yet. At some meetings, the ninjas must decide by majority if they attack or not, that is, they attack only if they are at least 64 and a majority wants to attack. Sensei has a majority protocol for this; in her jargon she says that the protocol *decides* the predicate $Q_2(n_Y, n_N) \Leftrightarrow n_Y + n_N \geq 64 \wedge n_Y > n_N$ (instead of the predicate $Q_1(n) \Leftrightarrow n \geq 64$ of the first protocol). She also has protocols for qualified majority (i.e., the ninjas attack only if at least $2/3$ of them are in favour, corresponding to the predicate $Q_2(n_Y, n_N) \Leftrightarrow 2n_Y \geq 3n_N$), and for whether the number of ninjas showing up is a multiple of 7, corresponding to $Q_3(n) \Leftrightarrow \exists k: n = 7k$.¹ In fact, Sensei has read [4], in which Angluin et al. introduced this kind of protocols and showed how to construct a

¹Ninjas are superstitious.

protocol for any predicate expressible in Presburger arithmetic. She has even read the improved version [6], which shows how to construct protocols slightly faster than those of [4] and, more importantly, with polynomially instead of exponentially many states in the size of the Presburger formula (assuming it is quantifier-free). However, these protocols are not robust. For example, the protocol of [6] for the predicate $x \geq 64$ is the first protocol of the introduction. Since these protocols are very dangerous in the presence of snipers, Sensei has the following question:

Is there a robust protocol for every Presburger predicate?

We do not know yet. In this paper we give a positive partial answer. We construct robust protocols for *threshold predicates* of the form $\sum_{i=1}^k a_i x_i \geq b$, where $a_1, \dots, a_k, b \in \mathbb{Z}$, and for *modulo predicates* of the form $\sum_{i=1}^k a_i x_i \equiv_c b$, where $a_1, \dots, a_k \in \mathbb{Z}$, $c \in \mathbb{N}$, and $0 \leq b \leq c - 1$.

Related work. The literature on fault-tolerance and robust distributed algorithms is very large, and so we consider only work on population protocols or closely related models. In [7], Delporte-Gallet *et al.* initiated the study of population protocols with essentially the same fault model as ours. The paper provides a construction that, loosely speaking, yields robust protocols with respect to a given number of failures, independent of the number of agents. The construction we give for modulo protocols is an immediate consequence of this idea. Delporte-Gallet and collaborators have published many other papers on fault-tolerant consensus and leader election algorithm in different kinds of message-passing networks.

There also exists previous work on other fault models. In [8], Guerraoui and Ruppert study community protocols, a model that extends population protocols by assigning unique identities to agents. They present a universal construction that, given a predicate in $\text{NSPACE}(\log n)$, outputs a community protocol deciding it. The construction is robust with respect to Byzantine failures of a constant number of agents. More recently, Alistarh *et al.* have studied robustness of population protocols and chemical reaction networks with respect to leaks [1, 2]. Loosely speaking, this is a failure model in which agents can probabilistically move to adversarially chosen states.

Structure of the paper. Section 2 introduces the syntax of population protocols and their semantics in the presence of a sniper. Section 3 formalizes the notion of robustness, and formally proves that the first and second of Sensei’s protocols are not robust and robust, respectively. Sections 4 and 5 give robust population protocols for all threshold and modulo predicates, respectively. Section 6 explains the problem with robustness of the standard construction for Boolean combinations.

2 Generalized Protocols and Population Protocols with Sniper

After some basic notations, we introduce the syntax and the semantics in the presence of a sniper of population protocols. For convenience, we introduce the model as a particular instance of a slightly more general model called generalized protocols.

Preliminaries: multisets and intervals. Given a finite set Q , a *multiset* is a function $M : Q \rightarrow \mathbb{N}$. We denote the set of multisets over Q by $\mathbb{N}^Q$. Multisets can be added, subtracted and compared componentwise. We define the *size* of a multiset M as $|M| := \sum_{q \in Q} M(q)$. The set of multisets of size k is denoted by $\mathbb{N}_k^Q$. The *support* of a multiset is $\llbracket M \rrbracket := \{q \in Q : M(q) \neq 0\}$. We use a set-like notation to write multisets explicitly: $\langle x, y, y, 3 \cdot z \rangle$ denotes the multiset M with $M(x) = 1$, $M(y) = 2$, $M(z) = 3$ and otherwise 0. When the set Q contains natural numbers, we write the numbers from Q in bold to differentiate them from their multiplicity: $\langle \mathbf{1}, 2 \cdot \mathbf{3} \rangle$.

We write $[a, b)$ to denote the set $\{x \in \mathbb{Z} : a \leq x < b\}$. We define the length of an interval as $\text{len}([a, b)) := b - a$.

Generalized Protocols: Syntax. A *generalized protocol* is a tuple: $\mathcal{P} = (Q, \delta, I, O)$ where

- Q is a finite set of states,
- $\delta \subseteq \mathbb{N}_2^Q \times \mathbb{N}_2^Q$ is the transition relation,
- $I \subseteq Q$ is the set of initial states,
- $O : \mathbb{N}^Q \rightarrow \{1, 0, \perp\}$ the output function.

Elements of $\mathbb{N}^Q$ are called *configurations* and elements of δ *transitions*. Configurations consist of *agents*, which are in a specific state $q \in Q$. We also call agents *ninjas*. Intuitively, the output function assigns to each configuration either a decision (1 or 0), or no decision ($\perp$). We let $p, q \mapsto p', q'$ denote that $(\langle p, q \rangle, \langle p', q' \rangle) \in \delta$. The *preset* and *postet* of a transition $t = (p, q \mapsto p', q')$ are $\text{PRE}(t) := \langle p, q \rangle$ and $\text{POST}(t) := \langle p', q' \rangle$. We call a transition t with $\text{PRE}(t) = \text{POST}(t)$ *silent*.

Additionally we assume that for every pair of states $p, q \in Q$, there exists a transition t with $\text{PRE}(t) = \langle p, q \rangle$. If the protocol does not define such a transition explicitly, we add the respective identity transition (such a transition is silent and does not change the behavior of the protocol).

Steps, snipes, and moves. Let $C, D \in \mathbb{N}^Q$ be two configurations.

- We write $C \rightarrow D$ if $C = D$ or there exists some transition t s.t. $\text{PRE}(t) \leq C$ and $D = C + \text{POST}(t) - \text{PRE}(t)$. We call $C \rightarrow D$ a *move* of the protocol.

A configuration C is *terminal* if there is no configuration $D \neq C$ such that $C \rightarrow D$.

- We write $C \hookrightarrow D$ if $C = D + S$ for some configuration S such that $|S| = 1$. We call $C \hookrightarrow D$ a *snipe*.
- We write $C \Rightarrow D$ if either $C \rightarrow D$ or $C \hookrightarrow D$, and call $C \Rightarrow D$ a *step*. That is, a step is either a move of the protocol, or a snipe.
- We let $\rightarrow^*$, $\hookrightarrow^*$, and $\Rightarrow^*$ denote the reflexive and transitive closure of $\rightarrow$, $\hookrightarrow$ and $\Rightarrow$, respectively.
- If $C \rightarrow^* D$ and $E \leq D$, we say that C *covers* E .

***i*-executions, *i*-reachability, fairness.** Fix $i \geq 0$. Loosely speaking, an *i*-execution is an infinite sequence of steps containing at most i snipes. Formally, an *i*-execution is an infinite sequence $\pi = \{C_j\}_{j \in \mathbb{N}}$ of configurations such that $C_j \Rightarrow C_{j+1}$ for every $j \geq 0$, and $C_j \hookrightarrow C_{j+1}$ for at most i values of j . The *output* of π is

$$\text{OUT}(\pi) := \begin{cases} 1 & \text{if } \exists n \in \mathbb{N} : \forall i \geq n : O(\pi(i)) = 1 \\ 0 & \text{if } \exists n \in \mathbb{N} : \forall i \geq n : O(\pi(i)) = 0 \\ \perp & \text{else} \end{cases}$$

An *i*-execution π is *fair* iff for every configuration $D \in \mathbb{N}^Q$ the following holds: if D can be reached from infinitely many configurations of π by executing sequences of moves, then D appears in π . It is easy to see that fairness can be equivalently defined as follows: if D can be reached *in one move* from infinitely many configurations of π , then D appears *infinitely often* in π . Formally, for every configuration D , if $\pi(j) \rightarrow D$ for infinitely many $j \in \mathbb{N}$, then $\pi(j) = D$ for infinitely many $j \in \mathbb{N}$.

Predicate decided by a generalized protocol. Fix $i \geq 0$. Given a configuration $C \in \mathbb{N}^Q$ we define its *i*-output, denoted $\text{OUT}_i(C)$, as follows:

$$\text{OUT}_i(C) := \begin{cases} 0 & \text{if } \forall \pi \text{ fair } i\text{-execution, } \pi(0) = C : \text{OUT}(\pi) = 0 \\ 1 & \text{if } \forall \pi \text{ fair } i\text{-execution, } \pi(0) = C : \text{OUT}(\pi) = 1 \\ \perp & \text{else} \end{cases}$$

A generalized protocol *i*-decides a predicate $\varphi : \mathbb{N}^I \rightarrow \{1, 0\}$ if $\text{OUT}_i(C) = \varphi(C)$ for every $C \in \mathbb{N}^I$.

Consensus output functions, population protocols. An output function $O : \mathbb{N}^Q \rightarrow \{1, 0\}$ is a *consensus function* if there is a partition of Q into two sets Q_+ and Q_- of *accepting* and *rejecting* states such that for every configuration C we have: $O(C) = 1$ if $\llbracket C \rrbracket \subseteq Q_+$, $O(C) = 0$ if $\llbracket C \rrbracket \subseteq Q_-$, and $O(C) = \perp$ otherwise. A generalized protocol is a *population protocol* if its output function is a consensus function.

Intuitively, if we interpret Q_+ and Q_- as the set of states in which ninjas have accept and reject *opinions*, respectively, then a configuration of a population protocol has output $b \in \{1, 0\}$ iff all ninjas currently have the same opinion, i.e., have currently reached a *consensus*, and $\perp$ otherwise.

Example 1. We restate the first protocol from the introduction using this formalism for an arbitrary threshold t :

Protocol PEBBLES.

States.	$Q = [0, t]$
Input.	$I = \{1\}$
Output.	$O(x) = 1$ if $x = t$, and $O(x) = 0$ otherwise
Transitions.	$\delta = \{(x, y) \mapsto (x + y, 0) : x, y \in Q \wedge x + y < t\} \cup \{(x, y) \mapsto (t, t) : x, y \in Q \wedge x + y \geq t\}$

3 Robustness

Recall the first protocol from the introduction. Even if the initial number of ninjas is 126, the sniper can still prevent the attack by taking one single ninja down. However, this is the case only because the sniper can intervene at any moment. Indeed, if the sniper can only intervene before the protocol starts, she must take at least 62 ninjas down to prevent the attack. We say that the initial configuration with 125 ninjas has an *initial tolerance* of 62, but a *global tolerance* of 1. It is easy to see that the initial tolerance for an arbitrary configuration with $n \geq 64$ is $n - 64$, while the global tolerance is $\lceil \frac{n}{63} \rceil - 2$. However, in the second protocol, the initial and global tolerance of any initial configuration with $n \geq 64$ ninjas coincide, and are equal to $n - 64$.

Observe that the initial tolerance of an initial configuration with n ninjas is $n - 64$ for *any* protocol deciding whether $n \geq 64$ holds. Intuitively, it gives an upper bound on how fault-tolerant a protocol for this predicate can be. On the contrary, the global tolerance depends on the protocol. This suggests to define the class of robust protocols as those protocols whose global tolerance is equal to the initial tolerance of all initial configurations. We proceed to formalize this notion.

Definition 1. Let P be a well-specified generalized protocol, and let $C \in \mathbb{N}^I$ be an initial configuration of P .

- The initial tolerance of C , denoted $\text{InTol}(C)$, is the largest number i such that $\text{OUT}_0(C) = \text{OUT}_0(D)$ for every configuration D with $C \hookrightarrow^i D$.
- The global tolerance of C , denoted $\text{Tol}(C)$, is the largest number i such that $\text{OUT}_i(C) = \text{OUT}_0(C)$.

P is robust in C if $\text{Tol}(C) = \text{InTol}(C)$. We call P robust if it is robust in every initial configuration.

We note a couple of useful facts about robustness:

Remark 1.

- Negating a population protocol via changing accepting states to be rejecting states and vice-versa, the protocol still remains robust. This is because neither the initial tolerance nor the global tolerance change.
- Sniping agents in initial states that have never interacted cannot change the output when considering snipes less than the initial tolerance. This follows directly from the definition: when sniping a agent in an initial state, it is as if the initial configuration was different to begin with.

3.1 Examples

We show that with our notion of robustness, the original population protocol for threshold predicates by Angluin et al. [3] is not robust.

Proposition 1. The protocol of [Example 1](#) is not robust for any $t \geq 3$.

Proof. Assume the protocol is robust. Let $q := t - 1 \in Q$ denote the last non-accepting state and consider the following fair 1-execution:

$$\langle (t+1) \cdot \mathbf{1} \rangle \rightarrow^* \langle \mathbf{q}, \mathbf{2}, (t-1) \cdot \mathbf{0} \rangle \hookrightarrow \langle \mathbf{2}, (t-1) \cdot \mathbf{0} \rangle \rightarrow \dots$$

The first configuration is an initial configuration with an initial tolerance of 1. Without sniping, it is accepted. However, the last configuration is terminal and rejecting. This contradicts the assumption that the protocol is robust. $\square$

Remark 2. *We can say even more about the robustness of this protocol. If the sniper is only permitted to snipe a single ninja, then we know that they can at worst snipe a ninja with $t - 1$ pebbles. Thus if the input configuration has $t - 1$ additional ninjas, then the output is still correct. In other words, if the initial tolerance of a configuration is i , then the global tolerance of the protocol is $\lfloor \frac{i}{t-1} \rfloor$.*

In this paper, we focus our attention on robust protocols and as such we do not need a fine-grained analysis of protocols that fail to be robust. But if one wants to analyze non-robust protocols, it is useful to define robustness with a parameter that is the ratio of initial and global tolerance.

We give the formal definition of Sensei's second protocol, which we call the TOWER protocol.

Protocol TOWER.

States.	$Q = [1, t]$
Input.	$I = \{1\}$
Output.	$O(x) = 1$ iff $x = t$
Transitions.	$\delta = \{(x, x) \mapsto (x, x+1) : x \in Q \wedge x < t\} \cup \{(x, t) \mapsto (t, t) : x \in Q, x < t\}$

Let us prove that the TOWER protocol is robust.

Proposition 2. *The TOWER protocol is robust for every t .*

Proof. Let $C \in \mathbb{N}^I$ be an initial configuration. If $|C| < t$, then $\text{OUT}_0(C) = 0$ and C cannot *cover* the accepting state, i.e. it cannot put an agent into t . Since the transition relation is monotonic ($C_1 \rightarrow^* C_2$ implies $C_1 + D \rightarrow^* C_2 + D$ for all $C_1, C_2, D \in \mathbb{N}^Q$), sniping cannot lead to reaching the accepting state. Therefore $\text{OUT}_i(C) = 0$ for all i .

If $|C| \geq t$, then we know that $|C| \geq t + \text{InTol}(C)$ by definition of the initial tolerance. To show $\text{InTol}(C) = \text{Tol}(C)$, we show that the protocol accepts any configuration with at least t agents.

First, we note that each transition increases the value of $\sum_{q \in Q} q \cdot C(q)$, so only finitely many transitions can be executed. Thus the protocol eventually reaches a terminal configuration D with $|D| \geq t$. If there is an agent in state t , then all other agents must be in state t as well — otherwise the second transition could be executed. So D is accepting in this case. If there is no agent in the t state, then we have at least t agents distributed among $t - 1$ states, so there is some state $x < t$ with at least two agents. But this is a contradiction to D being terminal, since those two agents can initiate a transition. $\square$

4 Robust Threshold Protocols

In this section we construct robust protocols for *threshold predicates*, that is, predicates of the form,

$$\varphi(x) = \left(\sum_{i=1}^n a_i \cdot x_i \geq t \right)$$

where $a_1, \dots, a_n, t \in \mathbb{Z}$. We proceed in three steps. [Section 4.1](#) constructs robust protocols for threshold predicates with positive coefficients a_i and an arbitrary threshold t . [Section 4.2](#) does the same for predicates with arbitrary coefficients and threshold 1; we call them *generalized majority* predicates, because they generalize the majority predicate $x - y \geq 1 \iff x - y > 0$. Finally, [Section 4.3](#) combines the protocols of the two previous sections to present protocols for arbitrary threshold predicates.

4.1 Threshold predicates with positive coefficients

In this section we study the case $a_i > 0$. We can wlog assume that $t \geq a_i$ for all i : If $t < 0$, then φ is equivalent to true, and we are done. Otherwise, since all coefficients $a_i > 0$, we replace every a_i by $\min(a_i, t)$ to obtain a new predicate φ' : Clearly if some ninja for a coefficient a_i which was decreased occurs, then both φ and φ' are automatically true. Since changing a coefficient which does not occur does not change satisfaction either, φ and φ' are equivalent.

The Black Ninjas have recently recruited a cohort of rookie ninjas. In battle, two veterans are as good as three rookies. When the ninjas meet, the question to decide is whether their battle power is at least 192 (the power of 64 veterans). In other words, the ninjas must conduct a protocol to decide if $3v + 2r \geq 192$, where v and r are the numbers of veterans and rookies, respectively.

The protocol of [3] for this case is an easy generalization of the protocol for $n \geq 64$. Each ninja brings a pouch, but veterans put three pebbles in it, while rookies only put two. Otherwise the protocol works as before: when ninjas interact, one gives her pebbles to the other, until two interacting ninjas observe that their joint number of pebbles is at least 192. The protocol fails to be robust, for the same reason as the previous one.

Sensei comes up with a generalization of the TOWER protocol. She visualizes again a TOWER with 192 floors. However, veterans now occupy three consecutive floors, e.g. floors 100, 101, and 102, and rookies occupy two consecutive floors, e.g. 101 and 102 (see [Figure 2](#)). If two ninjas that interact share at least one floor, then the ninja with the higher lowest floor moves one floor up. For example, if the two ninjas just mentioned interact, then the rookie moves one floor up, occupying now the floors 102 and 103 (if both ninjas have the same lowest floor, then any one of them moves up). The other ninja stays where she is. Initially, all veterans occupy the floors 0, 1, and 2, and all rookies occupy the floors 0 and 1.

Sensei's reasoning is similar to the previous case: eventually each floor is occupied by at most one ninja (although she has to think a bit to establish this). Therefore, a ninja eventually reaches the top floor iff $3v + 2r \geq 192$. As before, a ninja that reaches the top

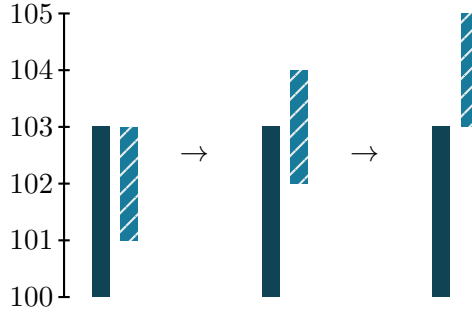


Figure 2: Two steps using the $\langle \text{step} \rangle$ transition between a veteran (dark blue) and a rookie (light blue, striped) in the INHOMTOWER protocol.

floor tells any other ninja she interacts with to move to the top floor too. We call this protocol INHOMTOWER, and proceed to define it formally.

Protocol INHOMTOWER.

The states of the protocol are intervals of natural numbers. For convenience we use semi-closed intervals $[s, e)$. A ninja in state $[s, e)$ occupies the floors $s, s + 1, \dots, e - 1$. The initial states are the intervals starting at zero. Formally we set

States. $Q = \{[s, s + a_i) : i \in [1, n] \wedge 0 \leq s \wedge s + a_i \leq t + 1\}$

Input. $I = \{[0, a_i) : i \in [1, n]\}$

Output. $O([s, e]) = 1$ iff $e = t + 1$

Transitions.

- For every two non-disjoint intervals $[s_1, e_1), [s_2, e_2) \in Q$ such that $s_1 \leq s_2$ and $e_1, e_2 < t + 1$ the second interval moves one step “upwards”:

$$[s_1, e_1), [s_2, e_2) \mapsto [s_1, e_1), [s_2 + 1, e_2 + 1) \quad \langle \text{step} \rangle$$

- For two intervals $[s_1, t + 1), [s_2, e_2) \in Q$ with $e_2 \leq t$, i.e. the first interval has reached the top, the second moves to the top as well.

$$[s_1, t + 1), [s_2, e_2) \mapsto [s_1, t + 1), [t + 1 - (e_2 - s_2), t + 1) \quad \langle \text{accum} \rangle$$

Theorem 1. *INHOMTOWER computes φ and is robust.*

Proof. Given an agent in state $[s, e) \in Q$, we say that it *occupies* levels $s, \dots, e - 1$.

Let $C \in \mathbb{N}^I$ denote an initial configuration and let $k := \text{InTol}(C)$. We show $\text{OUT}_0(C) = \varphi(C)$ (i.e. the protocol has the correct output on C) and $\text{OUT}_k(C) = \text{OUT}_0(C)$ (i.e. the protocol is robust on C).

Case 1: $\sum_{q \in Q} \text{len}(q)C(q) < t$. This implies $\varphi(C) = 0$. Here, we claim that no agent reaches a state $[s, t + 1]$ and $\langle \text{accum} \rangle$ is never executed, which implies $\text{OUT}_0(C) = 0$. To see that this is true, note the invariant that the set of levels in $[0, t]$ occupied by at least one agent is contiguous and contains level 0. Formally, for any D 0-reachable from C we have $\bigcup_{D(q) > 0} q = [0, i]$ for some i . We prove this invariant by induction. The base case follows from the definition of I , and for the induction step we use that $\langle \text{accum} \rangle$ is never executed, and that in transition $\langle \text{step} \rangle$ we have $s_2 \in [s_1, e_1]$, so no hole is created, and $s_1 \leq s_2$, so 0 remains occupied.

This proves $\text{OUT}_0(C) = \varphi(C)$. To show $\text{OUT}_k(C) = \text{OUT}_0(C)$ we again use a monotonicity argument similar to the proof of [Theorem 2](#). We have already shown that C cannot cover a state with $t + 1$ as the upper bound without snipes, and this extends to arbitrary k -executions.

Case 2: $\sum_{q \in Q} \text{len}(q)C(q) \geq t$. Note that any transition leaves $\sum_{q \in Q} \text{len}(q)C(q)$ invariant. Let D denote any configuration k -reachable from C .

If $\sum_q \text{len}(q)D(q) < t$, then this observation would imply that it is possible to snipe k agents from C (without executing any moves) and end up in a configuration with output 0. But this contradicts $k = \text{InTol}(C)$.

Again similar to the proof of [Theorem 2](#), transitions $\langle \text{step} \rangle$ and $\langle \text{accum} \rangle$ can only be executed finitely often, so we assume wlog that D is terminal. We now argue that D has output 1. If level t is occupied, i.e. in D there is some agent in a state $[s, t + 1]$, then all other agents must be in such a state as well (otherwise $\langle \text{accum} \rangle$ could be executed) and the protocol accepts. Otherwise we use $\sum_{q \in Q} \text{len}(q)D(q) \geq t$: an agent in state q occupies $\text{len}(q)$ levels and only levels in $[0, t - 1]$ are occupied, so at least one level is occupied by two agents. However, these agents could execute $\langle \text{step} \rangle$, contradicting that D is terminal. $\square$

4.2 Threshold predicates with threshold 1

We design a robust protocol for predicates of the form

$$\varphi(x) = \left(\sum_{i=1}^n a_i \cdot x_i \geq 1 \right)$$

Since they are generalizations of the majority predicate $x - y \geq 1 \iff x - y > 0$, we call them *generalized majority* predicates. In this section, we first introduce weak population protocols. Intuitively, in a weak consensus protocol not all states must be accepting or rejecting, but can also be neutral. Then we present a very simple and yet robust weak population protocol for generalized majority. Finally, we present a conversion procedure that transforms a weak population protocol into a population protocol for the same predicate that also preserves robustness.

4.2.1 Weak population protocols for generalized majority

Recall that population protocols reach a decision by *consensus*. States can be split into accepting or rejecting. A configuration is a *consensus* if either all ninjas are in accepting

states or all ninjas are in rejecting states. The output function assigns a decision to each consensus configuration, and only to them. We present *weak population protocols* in which states can also be *neutral*. Intuitively, ninjas in neutral states have no opinion. A configuration is a *weak consensus* if no two ninjas have conflicting opinions. The output function assigns a decision to each consensus configuration, and only to them, as follows: accept if at least one ninja wants to accept and no ninjas wants to reject, and reject otherwise. Observe that, in particular, if no ninja wants to accept, then the decision is reject.

Definition 2. An output function $O: \mathbb{N}^Q \rightarrow \{1, 0\}$ is a *weak consensus function* if there is a partition of Q into three sets Q_+ , Q_+ , Q_- of accepting, neutral, and rejecting states such that for every configuration C :

- $O(C) = 1$ if $\llbracket C \rrbracket \subseteq Q_+ \cup Q_-$ and $\llbracket C \rrbracket \cap Q_+ \neq \emptyset$;
- $O(C) = 0$ if $\llbracket C \rrbracket \subseteq Q_- \cup Q_-$; and
- $O(C) = \perp$ otherwise (that is, $O(C) = \perp$ if $\llbracket C \rrbracket \cap Q_+ \neq \emptyset \neq \llbracket C \rrbracket \cap Q_-$).

A generalized protocol $\mathcal{P} = (Q, \delta, I, O)$ is a *weak population protocol* if O is a weak consensus function, and every 1-consensus is stable.

Notice that the output function is determined by the partition Q_+ , Q_+ , Q_- . Let us now give weak population protocols for generalized majority predicates. We consider the predicate $x_1 - 2x_2 \geq 1$, which is already representative. It corresponds to the following situation. Sensei wants that the ninjas conduct a vote to decide whether they want to attack or not. The ninjas will only attack if more than 2/3 of them vote for it. Letting x_1 and x_2 be the number of ninjas in favor of and against attacking, respectively, the ninjas will attack if $x_1/(x_1 + x_2) > 2/3$ or, equivalently, $x_1 - 2x_2 \geq 1$. Intuitively, a negative vote has the same effect as two positive votes.

Sensei asks the ninjas to bring a pouch containing one *positive pebble* if they want to attack, and two *negative pebbles* otherwise. If two ninjas with pebbles of the same kind interact, nothing happens. If the pebbles are of opposite kinds, one ninja gives her pebbles to the other. However, when a positive and a negative pebble touch each other, they magically disappear, and so after an interaction between two ninjas with a positive and b negative pebbles one ninja has $|a - b|$ pebbles and the other 0. We visualize this protocol in [Figure 3](#).

The possible states of the protocol are $\{1, 0, -1, -2\}$, which gives the number of pebbles in the pouch and their kind. The partition of states is the natural one: $Q_+ = \{1\}$, $Q_+ = \{0\}$, and $Q_- = \{-1, -2\}$. The only non-silent transitions are $1, -2 \mapsto 0, -1$, and $1, -1 \mapsto 0, 0$. Observe that, in particular, $1, 1 \mapsto 2, 0$ or $-2, -2 \mapsto -4$ are *not* transitions, because the pebbles of the interacting ninjas are of the same kind.

Let us informally argue that the protocol is correct and robust. Consider a configuration C in which x_1 ninjas want to attack, and x_2 ninjas do not. Observe first that the total number of pebbles never increases, and it decreases whenever two ninjas carrying pebbles of opposite kinds interact. Therefore, starting from C the protocol eventually reaches a terminal configuration C' in which all remaining pebbles (if any) are positive or all

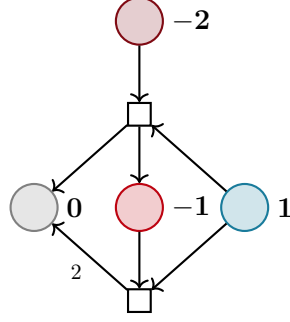


Figure 3: Weak population protocol for $x_1 - 2x_2 \geq 1$. Red states are rejecting, the blue state is accepting and the gray state is neutral.

are negative. In the first case only states of $Q_+ \cup Q_-$ are populated, and in the second only states of $Q_- \cup Q_-$. So C' is a weak consensus. Further, it is easy to see that the cases occur when $x_1 - 2x_2 \geq 1$ and $x_1 - 2x_2 < 1$, respectively. To prove robustness, observe that the argument above holds for any configuration C , and so for the sniper all configurations are equally good.

Let us now give the precise definition of the protocol for the predicate $\sum_{i=1}^n a_i \cdot x_i \geq 1$. Let a_{min}, a_{max} be the minimum and maximum of $\{a_1, \dots, a_n\}$. We assume $a_{min} < 0 < a_{max}$, the case of predicates in which all coefficients have the same sign was already considered in the previous section. The protocol is:

Protocol GENMAJORITY.

- States.** $Q = [a_{min}, a_{max}]$
- Input.** $I = \{a_1, \dots, a_n\}$
- Output.** Given by $Q_+ = [1, a_{max}]$, $Q_- = \{0\}$, and $Q_- = [a_{min}, -1]$.
- Transitions.** $\delta = \{x, y \mapsto x + y, 0 : x, y \in Q \wedge x < 0 < y\}$

4.2.2 From weak population to population protocols

In order to use weak population protocols in practice, we provide an algorithm that given a weak population protocol deciding a predicate, under the assumption that the protocol is *terminating*, i.e. every fair execution terminates, outputs a population protocol for the same predicate. Further, the algorithm preserves robustness.

The construction to obtain a population protocol $\mathcal{P}'$ from a weak population protocol $\mathcal{P}$ is rather simple: Every ninja remembers a state of $\mathcal{P}$, and in fact runs the protocol $\mathcal{P}$ as normal. In addition, neutral ninjas A , i.e. in a state $q \in Q_-$, get an output bit $\{+, -\}$, which they set as follows: Whenever they meet a ninja B with an active opinion, i.e. with state $p \in Q_+ \cup Q_-$, A updates her bit to the opinion of B . Since by definition of weak consensus eventually only one opinion remains, every neutral ninja eventually gets the opinion of the remaining ninja. A slight difficulty, where we need the assumption of termination of $\mathcal{P}$, is the case of only neutral ninjas remaining: We add a transition which takes neutral ninjas with output bits set to $+$ and $-$ respectively and sets both bits to $-$.

Construction 1. Given a weak population protocol $\mathcal{P} = (Q, \delta, I, O)$ with partition Q_+, Q_-, Q_+ , we construct a population protocol $\mathcal{P}' = (Q', \delta', I', O')$:

Protocol WEAKCONVERT.

States. $Q' = Q_+ \cup Q_- \cup Q_+ \times \{+, -\}$

Input. $I' = \{q : q \in I \cap (Q_+ \cup Q_-)\} \cup \{(q, -) : q \in I \cap Q_+\}$

Output. $O'(q') = \begin{cases} 1 & \text{if } q' \in Q_+ \text{ or } q' = (q, +) \text{ with } q \in Q_+ \\ 0 & \text{if } q' \in Q_- \text{ or } q' = (q, -) \text{ with } q \in Q_+ \end{cases}$

Transitions.

- First we need a transition to simulate $\mathcal{P}$, given agents in states p, q in $\mathcal{P}'$, they 1) disregard their sign if they are in Q_+ , 2) perform δ on the so obtained projected states p_{pr}, q_{pr} , and 3) read a negative sign if they end in Q_+ .

Formally, let $pr: Q' \rightarrow Q, q' \mapsto q$ if $q' \in Q_+ \cup Q_-$ and $q' \mapsto q$ if $q' = (q, s) \in Q_+ \times \{+, -\}$. Also let $inj: Q \rightarrow Q', q \mapsto q$ if $q \in Q_+ \cup Q_-$ and $q \mapsto (q, -)$ otherwise. For every non-silent transition $(p_{pr}, q_{pr} \mapsto p'_{pr}, q'_{pr}) \in \delta_{\mathcal{G}}$, we add for all states p, q with $pr(p) = p_{pr}, pr(q) = q_{pr}$, the following transition:

$$p, q \mapsto inj(p'_{pr}), inj(q'_{pr}) \quad \langle \text{derived} \rangle$$

- Additionally, for $q_+ \in Q_+$ and $q_- \in Q_-$ we add the following transition:

$$q_+, (q_-, -) \mapsto q_+, (q_-, +) \quad \langle \text{witnessPos} \rangle$$

- Dually, for $q_- \in Q_-$ and $q_+ \in Q_+$ we add the following transition:

$$q_-, (q_+, +) \mapsto q_-, (q_+, -) \quad \langle \text{witnessNeg} \rangle$$

- And for every $q, q' \in Q_+$ we add:

$$(q, -), (q', +) \mapsto (q, -), (q', -) \quad \langle \text{convince} \rangle$$

Given an execution π on $\mathcal{P}'$, we define the induced execution $\tilde{\pi}$ on $\mathcal{P}$ via $\tilde{\pi}(i) = pr(\pi(i))$ where $pr((q, s)) = q$. Moving from π to $\tilde{\pi}$ intuitively replaces $\langle \text{derived} \rangle$ by their respective transition and $\langle \text{witnessPos} \rangle / \langle \text{witnessNeg} \rangle / \langle \text{convince} \rangle$ by silent identity transitions. If π is fair, $\tilde{\pi}$ is also fair.

In the following, a positive voter is a ninja with a state in Q_+ , a negative voter a ninja with a state in Q_- and an active voter is either a positive or negative voter. Essentially there is only one type of configuration of $\mathcal{P}'$ where assigning the output as above will fail: If there are no active voters, and neither is a ninja in $Q_+ \times \{-\}$ to convince the others. Hence we call a configuration C *dangerous* if $\llbracket C \rrbracket \subseteq Q_+ \times \{+\}$, and *harmless* otherwise.

Lemma 1. Let C'_0 be a harmless configuration of $\mathcal{P}'$ such that all fair executions starting at $pr(C'_0) \in \mathbb{N}^Q$ terminate with output b in $\mathcal{P}$.

Then all fair executions starting at C'_0 terminate with output b in $\mathcal{P}'$.

Proof. Let $\pi = (\pi_0, \pi_1, \dots)$ be a fair execution in $\mathcal{P}'$ starting at C'_0 . We have to show that π has output b . First consider the induced execution $\tilde{\pi}$ in $\mathcal{P}$. Since $\mathcal{P}$ is terminating and $\tilde{\pi}$ is fair, $\tilde{\pi}$ terminates in some configuration C'_{pr} . Clearly using $\langle \text{witnessPos} \rangle / \langle \text{witnessNeg} \rangle$ and potentially $\langle \text{convince} \rangle$, π then also terminates in some configuration C' , and we know that $\text{pr}(C') = C'_{pr}$. Because $\tilde{\pi}$ is also fair, the output of C'_{pr} is b . If an active voter exists in C'_{pr} and hence in C' , it will eventually convert everyone to the correct output via the $\langle \text{witnessPos} \rangle / \langle \text{witnessNeg} \rangle$ transitions, and hence π again has output b .

If C' does not contain an active voter, then C'_{pr} has output 0 by definition, and we have to show the same for the execution π . We first show that the configuration C' contains an agent in $Q_{=} \times \{-\}$. We prove this via case distinction:

Case 1: No $\langle \text{derived} \rangle$ transition ever occurred. Then we have $\pi_i(Q_{+}) = 0$ for all i , and since C'_0 is harmless, initially an agent was in $Q_{=} \times \{-\}$. The only transition which could remove the agent from $Q_{=} \times \{-\}$, namely $\langle \text{witnessPos} \rangle$, was never enabled.

Case 2: $\langle \text{derived} \rangle$ did occur. Then consider the configuration π_i directly after the last occurrence. It contains an agent in $Q_{=} \times \{-\}$ by definition of $\langle \text{derived} \rangle$. We can now argue as in case 1.

It follows that $\langle \text{convince} \rangle$ moves all agents from $Q_{=} \times \{+\}$ to $Q_{=} \times \{-\}$, and π terminates with output 0. $\square$

Theorem 2. *Given a terminating weak population protocol $\mathcal{P}$ deciding a predicate φ , the above construction outputs a population protocol $\mathcal{P}'$ deciding φ . Additionally, if $\mathcal{P}$ is robust, then $\mathcal{P}'$ also is.*

Proof. Applying Lemma 1 to all initial configurations, we conclude that $\mathcal{P}'$ decides the same predicate φ as $\mathcal{P}$. Observe that initial configurations are harmless since agents start in $Q_{=} \times \{-\}$ by definition.

It remains to prove that $\mathcal{P}'$ is robust. Hence let $C'_0 \in \mathbb{N}^{I'}$ be an initial configuration. We have to show that $\text{Tol}(C'_0) = \text{InTol}(C'_0)$. Hence let π be a fair i -execution starting at C'_0 with $i = \text{InTol}(C'_0)$. We have to show that π has output $\varphi(C'_0)$.

Observe that since $\mathcal{P}$ is terminating, $\mathcal{P}'$ is also terminating. Hence π' eventually becomes constant, we call the reached terminal configuration C'' . Even if an agent was sniped earlier in π , we can wlog assume it was sniped directly before C'' , and simply did not participate in transitions before that. I.e. there exists $C' \in \mathbb{N}^{Q'}$ such that $C'_0 \rightarrow^* C'$ and $C' \hookrightarrow^i C''$. To show that π has output $\varphi(C'_0)$, it suffices to show that C'' has output $\varphi(C'_0)$. We want to apply Lemma 1 on the configuration C'' , hence we have to check its assumptions.

First, since $\mathcal{P}$ and $\mathcal{P}'$ compute the same predicate, we have $\text{InTol}_{\mathcal{P}'}(C'_0) = \text{InTol}_{\mathcal{P}}(\text{pr}(C'_0))$. Since $\mathcal{P}$ is robust, and the projected execution $\tilde{\pi}$ contains at most $i = \text{InTol}_{\mathcal{P}}(\text{pr}(C'_0))$ snipes, $\tilde{\pi}$ has output $\varphi(C'_0)$. Therefore $\text{pr}(C'')$ has output $\varphi(C'_0)$. Since $\text{pr}(C'')$ is terminal, this shows the assumption about all fair executions of the projection having correct output. It remains to prove that C'' is harmless.

Assume for contradiction that C'' is dangerous. In particular $\text{pr}(C'')$ is rejected. Similar to the case reasoning in Lemma 1, observe that the only transition outputting agents in $Q_{=} \times \{+\}$ is $\langle \text{witnessPos} \rangle$. Hence to reach a dangerous configuration, in some configuration

between C_0 and C' some agent must have been in Q_+ and got sniped. We denote the latest (i.e. closest to C') configuration that has a positive voter by C_+ .

Note that no agent in C'' participated in a transition since C_+ , since after C_+ no positive voters exist and hence all enabled transitions set the output bit to $-$. We now consider the configuration $C''_+ = C'' + \{q_+\}$ where q_+ is a positive voter in C_+ . We can reach C''_+ from C_+ via sniping exactly those agents that are sniped from C' to C'' except for the positive voter. Since C'' does not contain negative voters, C''_+ does not, and hence C''_+ is an accepting consensus. By assumption on weak population protocols, any accepting consensus is stable, hence C''_+ is accepted. Hence $\text{pr}(C'_0)$ has an accepting execution via C''_+ , and a rejecting execution via C'' , despite both executions sniping $\leq \text{InTol}(\text{pr}(C'_0))$ many agents. Contradiction to robustness of $\mathcal{P}$. $\square$

Example 2. We now apply this construction to the protocol that we gave above for $x_1 - 2x_2 \geq 1$ and visualized in [Figure 3](#). We write $+0$ and -0 instead of $(0, +)$ and $(0, -)$ for clarity.

Protocol SIGNEDNUMBERS.

States.	$Q = \{1, -1, -2, +0, -0\}$		
Input.	$I = \{-2, 1\}$		
Output.	$O(q) = \begin{cases} 1 & \text{if } q = 1 \text{ or } q = +0 \\ 0 & \text{if } q = -1 \text{ or } q = -2 \text{ or } q = -0 \end{cases}$		
Transitions.			
	$-2, \quad 1$	$\mapsto \quad -1, \quad +0$	coming from $\langle \text{derived} \rangle$
	$-1, \quad 1$	$\mapsto \quad -0, \quad -0$	coming from $\langle \text{derived} \rangle$
	$-0, \quad 1$	$\mapsto \quad +0, \quad 1$	coming from $\langle \text{witnessPos} \rangle$
	$+0, \quad -2$	$\mapsto \quad -0, \quad -2$	coming from $\langle \text{witnessNeg} \rangle$
	$+0, \quad -1$	$\mapsto \quad -0, \quad -1$	coming from $\langle \text{witnessNeg} \rangle$
	$+0, \quad -0$	$\mapsto \quad -0, \quad -0$	coming from $\langle \text{convince} \rangle$

4.3 Arbitrary threshold predicates

In this subsection we again consider the threshold predicate from the beginning of this section:

$$\varphi(x) = \left(\sum_{i=1}^n a_i \cdot x_i \geq t \right)$$

Now we also allow for $a_i, t \in \mathbb{Z}$. We again will create a tower-esque protocol, however this time it will be a weak population protocol. Ninjas representing inputs with negative coefficients will have to cancel positive ninjas. We assume $t \geq 1$ wlog, since we can rewrite $\sum_{i=1}^n a_i \cdot x_i \geq t$ as $\sum_{i=1}^n (-a_i) \cdot x_i \leq -t$, which is the negation of $\sum_{i=1}^n (-a_i) \cdot x_i \geq -t + 1$. Negation preserves robustness, see [Remark 1](#).

Protocol INHOMTOWERCANCEL.

States are intervals of integers as well as non-positive integers. We identify 0 with all empty intervals. Initial states are intervals starting at zero as well as negative integers.

States. $Q = Q_{\text{TOWER}} \cup Q_{\text{CANCEL}} \cup \{0\}$ where
 $Q_{\text{TOWER}} = \{[s, e) : 0 \leq s < e \leq T\}$
 $Q_{\text{CANCEL}} = \{x : \min_i a_i \leq x < 0\}$
 $T = \max\{t, a_1, \dots, a_n\}$

Input. $I = \{[0, a_i) : a_i > 0\} \cup \{a_i : a_i < 0\}$

Output. Given by the following partition:
 $Q_+ = \{[s, e) \in Q_{\text{TOWER}} : t \leq e\},$
 $Q_- = \{[s, e) \in Q_{\text{TOWER}} : e < t\} \cup \{0\},$
 $Q_- = Q_{\text{CANCEL}}$

Transitions.

- For every two non-disjoint intervals $[s_1, e_1), [s_2, e_2) \in Q_{\text{TOWER}}$ with $e_1, e_2 < T$ and $s_1 \leq s_2$ the transition

$$[s_1, e_1), [s_2, e_2) \mapsto [s_1, e_1), [s_2 + 1, e_2 + 1) \quad \langle \text{step} \rangle$$

- When a negative agent meets an agent above level t , they both reduce their absolute value by one. For all $0 \leq s < e \leq T$ with $t \leq e$ and $x \in Q_{\text{CANCEL}}$ the transition

$$[s, e), x \mapsto [s, e - 1), x + 1 \quad \langle \text{cancel} \rangle$$

note that we identify empty intervals i.e. $[s, s)$ with the state 0.

Theorem 3. *INHOMTOWERCANCEL computes φ and is robust.*

Proof. This proof is similar to that of [Theorem 1](#). Given an agent in state $[s, e) \in Q_{\text{TOWER}}$, we say that it *occupies* levels $s, \dots, e - 1$.

Let $C_0 \in \mathbb{N}^I$ be an initial configuration and let $k = \text{InTol}(C_0)$. Additionally, let $\text{Sum}(C) := \sum_{q \in Q_{\text{TOWER}}} \text{len}(q)C(q) + \sum_{q \in Q_{\text{CANCEL}}} qC(q)$ for any configuration C . Observe that any transition leaves Sum invariant. Since $\text{Sum}(C) \leq t$ is φ , initial tolerance ensures that sniping does not change $\text{Sum}(C) \leq t$.

Additionally, we note that the protocol produces an output for every fair n -execution for arbitrary n . Since eventually no agent reaches a level above or equal to t , or no negative states contain agents. We now show $\text{OUT}_0(C_0) = \varphi(C_0)$ and $\text{OUT}_k(C_0) = \text{OUT}_0(C_0)$.

Case 1: $\text{Sum}(C_0) < t$. This implies $\varphi(C_0) = 0$. For contradiction, we consider a fair i -execution starting at C_0 that is accepting with minimal $i \leq k$. Since the protocol is bounded, we can equivalently consider configurations $C_0 \rightarrow^* C \hookrightarrow^* D \rightarrow^* C_{\text{term}}$ where C_{term} is terminal.

We now show that we can assume that the snipes from C to D do not snipe agents that are in Q_{TOWER} . Let Δ denote the agents in Q_{TOWER} that were sniped in $C \hookrightarrow^* D$. Now consider $C' = D + \Delta$: we have $C \hookrightarrow^* C' \rightarrow^* C_{\text{term}} + \Delta$. The last configuration is again accepting, since it does not contain any agents in Q_{CANCEL} . While it is not terminal,

it cannot become rejecting, since agents in Q_{CANCEL} can never again exist. If $|\Delta| > 0$, then we have a smaller counterexample.

In the same manner as in the proof of [Theorem 1](#), we show that the set of occupied levels in a configuration D k -reachable from C without sniping in Q_{TOWER} is contiguous and contains 0. We show by induction that $\bigcup_{D(q)>0} q = [0, i]$ for some i . The base case follows from the definition of I , in the step case we must consider the two transitions: [⟨step⟩](#) does not introduce holes, since $s_2 \in [s_1, e_1)$ and $s_1 \leq s_2$, so 0 remains occupied. [⟨cancel⟩](#) either reduces the upper bound of an interval at or above t by one, this also does not create a hole and leaves 0 occupied.

Now we consider the output of C_{term} . If there are agents in Q_{CANCEL} , then the levels $[t, T)$ cannot be occupied, since otherwise [⟨cancel⟩](#) could be executed. If no agents are in Q_{CANCEL} , then we also know that $[t, T)$ cannot be occupied, since $\text{Sum}(C_{\text{term}}) = \sum_{q \in Q_{\text{TOWER}}} \text{len}(q)C_{\text{term}}(q) < t$ and the occupied levels are contiguous and include 0.

Case 2: $\text{Sum}(C_0) \geq t$. This implies $\varphi(C_0) = 1$. Let D be a configuration k -reachable from C_0 .

If $\text{Sum}(D) < t$, then by the observation above it would be possible to snipe k agents in C_0 and obtain an initial configuration with $\text{Sum} < t$. By the previous case, it would reject, thus contradicting that $k = \text{InTol}(C_0)$.

We again, similar to the proofs [Theorem 2](#) and [Theorem 1](#) use the fact that [⟨step⟩](#) and [⟨cancel⟩](#) can only be executed finitely often. So wlog D is terminal. We show that D has output 1: if level t or any above is occupied, then since D is terminal, there cannot exist any agent in D that is in a state from Q_{CANCEL} . Therefore D has output 1. In the case that no agent occupies level t or above, we know from $\text{Sum}(D) \geq t$ that $\sum_{q \in Q_{\text{TOWER}}} \text{len}(q)D(q) \geq t$ and thus at least one level is occupied by two agents. This means that [⟨step⟩](#) could be executed contradicting that D is terminal. $\square$

Corollary 1. *Giving INHOMTOWERCANCEL as input to [Construction 1](#) we obtain a robust population protocol computing φ .*

Proof. Clearly INHOMTOWERCANCEL is terminating, and [Theorem 3](#) shows it is robust. Every accepting consensus is stable: Namely no agent is in Q_{CANCEL} since these are negative voters, wherefore [⟨cancel⟩](#) is permanently disabled. On the other hand [⟨step⟩](#) can only cause additional agents to become positive voters. Hence we can apply [Theorem 2](#). $\square$

5 Robust Modulo Protocols

Throughout this section, fix a modulo predicate not equivalent to true or false:

$$\varphi(x) = \left(\sum_{i=1}^n a_i \cdot x_i \bmod m \geq t \right)$$

With $a_i, t \in \mathbb{Z}$ and $m \in \mathbb{N}$. Without loss of generality, we can assume that $0 < a_i, t < m$, because the sum is calculated modulo m .

When the Black Ninjas attack fortresses of the Dark Powers on nights with a full moon, they only do so if the number of ninjas modulo 7 is equal to 4, 5 or 6. Therefore,

Sensei is searching for a robust population protocol for $x \bmod 7 \geq 4$. Sensei finds an important difference between threshold and modulo predicates: Threshold predicates allow for arbitrarily large initial tolerance, for modulo predicates however there exists an upper bound on the initial tolerance.

Proposition 3. *For any population protocol computing φ , we have for any initial configuration $C \in \mathbb{N}^I$ that $\text{InTol}(C) < m$.*

Proof. Using at most $m - 1$ snipes, the value of the sum can be changed by at least $m - 1$, thus allowing to change the output. $\square$

Using this fact, we will first construct a protocol for sufficiently large inputs in [Section 5.1](#). Then we will combine that protocol in [Section 5.2](#) with an `INHOMTOWER` and a protocol for small inputs to obtain a robust population protocol for all inputs.

5.1 A Modulo Protocol for Big Inputs

After proving [Proposition 3](#), Sensei comes up with a robust population protocol that can handle sufficiently large initial configurations. The sniper snipes at most $m - 1$ ninjas, allowing Sensei to take advantage of a principle common with error-correcting codes: to achieve a fault tolerance of r , $2r + 1$ copies of a fault intolerant code are used in parallel. Then one can decide the correct value via majority. Sensei applies this principle to protocols: she thinks that it will be sufficient to have $2(m - 1) + 1 = 2m - 1$ copies of any protocol for a modulo predicate running in parallel. For combining with small inputs, which will become clear later, we need an additional copy, so we will use $2m$ copies in total.

To compute $x \bmod 7 \geq 4$, Sensei gives the following instructions to the ninjas: Each ninja must bring 14 differently colored pouches, each containing a magic pebble. Whenever 7 magic pebbles are in the same pouch, they annihilate each other. Thus the pebbles are added modulo 7. In the beginning, each ninja must choose a color. After doing so, they become a *leader* for that color. When a leader and any other ninja that is not a leader for the same color interact, the leader steals all pebbles from the other's pouch with the chosen color. When two leaders for the same color interact, one of them retires and gives all the pebbles of that color to the other. The retired leader then again chooses a color where she still has some pebbles left and can again become a different leader. If she has no pebbles left, she remains a non-leader. Additionally, every ninja remembers for every color the number of pebbles in the pouch of the last seen leader of that color. The ninjas decide to attack via majority: if a majority of leaders agree that there at least 4 pebbles in their pouch, then they attack.

Sensei argues that the protocol works for sufficiently large configurations (where $x \geq 14$): every ninja can at most be a leader for a given color once. At least one leader for every color will remain and since there are more than 14 ninjas, every color will have a leader. Every leader correctly computes the value $x \bmod 7$ via the magic pebbles and without a sniper, they all will agree. In the case of a sniper, for every snipe at most one

color can be disturbed. Since a sniper can at most snipe 6 times, the majority of 14 leaders will remain correct.

We now give the formal definition of the BIGMODULO population protocol, where we write $\mathbb{1}_A$ for the indicator function of the set A :

Protocol BIGMODULO.

States. $Q = [0, 2m] \times \mathbb{Z}_m^{2m} \times \{0, 1\}^{2m}$
Input. $I = \{(0, a_i \cdot \mathbb{1}_{[1, 2m]}, 0) : 1 \leq i \leq n\}$
Output. $O(_, _, r) = 1 \iff |\{j : r_j = 1\}| > m$
Transitions.

- We need to distribute the agents in the different copies. For this, we use non-determinism, for all $(0, v, r), q \in Q$ and $1 \leq i \leq 2m$ with $1 \leq v(i)$ we add the following transition:

$$(0, v, r), q \mapsto (i, v, r), q \quad \langle \text{distrib} \rangle$$

- When two agents meet that are in different copies, both steal the tokens from their respective copy from the other. For all $(i, v, r), (j, w, s) \in Q$ with $1 \leq i, j \leq 2m$ and $i \neq j$ we add the following transition:

$$(i, v, r), (j, w, s) \mapsto (i, v', r), (j, w', s) \quad \langle \text{steal} \rangle$$

Where

$$v' = v - v_j \cdot \mathbb{1}_{\{j\}} + w_i \cdot \mathbb{1}_{\{i\}} \quad w' = w - w_i \cdot \mathbb{1}_{\{i\}} + v_j \cdot \mathbb{1}_{\{j\}}$$

- When two agents from the same copy meet, one of them takes all tokens from the other and the other retires. For all $(i, v, r), (i, w, s) \in Q$ with $1 \leq i \leq 2m$ we add the following transition:

$$(i, v, r), (i, w, s) \mapsto (i, v + w_i \cdot \mathbb{1}_{\{i\}}, r), (0, w - w_i \cdot \mathbb{1}_{\{i\}}, s) \quad \langle \text{retire} \rangle$$

- Lastly we need a transition that updates the last component to keep track of the overall output of the protocol. For all $(i, v, r), (j, w, s) \in Q$ with $1 \leq i \leq 2m$ and $0 \leq j \leq 2m$ we add the following transition:

$$(i, v, r), (j, w, s) \mapsto (i, v, r'), (j, w, s') \quad \langle \text{result} \rangle$$

Where

$$r'(l) = \begin{cases} v_i \geq t & \text{if } l = i \\ r(l) & \text{else} \end{cases} \quad s'(l) = \begin{cases} v_i \geq t & \text{if } l = i \\ s(l) & \text{else} \end{cases}$$

We call agents with a non-zero first component *leaders*, and call the j -th copy of $\mathbb{Z}^m$ the j -th subprotocol.

Proposition 4. *For every initial configuration $C_0 \in \mathbb{N}^I$ with $|C_0| \geq 2m$, **BIGMODULO** decides φ .*

Proof. Let C_0 be an initial configuration. We first claim that every subprotocol will eventually have exactly one associated leader.

Proof of claim: When two leaders meet via $\langle \text{retire} \rangle$, only one of them remains a leader and the other gives away their tokens in that subprotocol. Since $\langle \text{distrib} \rangle$ only makes agents leaders that have tokens in the respective subprotocol, that agent will never again be a leader for that subprotocol. Since we have at least $2m$ agents, every subprotocol will eventually end up with a leader.

Now it is easy to see that each leader will eventually hold all the tokens in their respective subprotocol. Namely they accumulate them via $\langle \text{steal} \rangle$.

Since the j -th sum $\text{Sum}_j(C) := \sum_{q=(i,v,r) \in Q} v(j) \cdot C(q)$ is invariant modulo m , the leader for every subprotocol has exactly $\text{Sum}_j(C_0) \bmod m$ many tokens, hence obtaining the correct output. This output is then broadcasted via the $\langle \text{result} \rangle$ transition to every non-leader. $\square$

Proposition 5. *For every initial configuration $C_0 \in \mathbb{N}^I$ with $|C_0| \geq 3m$, **BIGMODULO** is robust.*

Proof. Let C_0 be such an initial configuration. First observe that even after sniping we then have at least $2m$ agents left. Intuitively the argument will be along the following lines: Case 1: If an agent has not interacted in a copy j yet, then sniping it falls into the initial tolerance. Case 2: If an agent has value 0 in a copy, it can also be sniped without issue. Case 3: Neither case 1 nor 2 is the case. We have to show that for every agent, case 3 happens for at most one copy. Then the number of copies damaged is at most the number of sniped agents, i.e. at most $m - 1$, and the majority of copies remains correct.

Hence let $\rho = (C_0, C_1, \dots)$ be a fair k -execution with $k = \text{InTol}(C_0)$. For every agent a we define

$$J(a, n) := \{j \in [1, 2m] : \begin{array}{l} a \text{ has interacted in the } j\text{-th copy of } \mathbb{Z}^m \text{ before step } n \\ \wedge v_j(a) \neq 0 \end{array}\}$$

$$L(a, n) := \begin{cases} \{j\} & \text{if } a \text{ is a leader in the } j\text{-th copy of } \mathbb{Z}^m \text{ at step } n \\ \emptyset & \text{else} \end{cases}$$

We claim that if C_n fulfills that for all agents $J(a, n) \subseteq L(a, n)$, then also C_{n+1} does. Comparing with the outline, this states exactly that case 3 occurs only for the copy j in which agent a is the leader. To see that this property is actually inductive, we inspect every transition: $\langle \text{steal} \rangle$ and $\langle \text{retire} \rangle$ set the value of agent a in a copy j to 0 on the non-leader, $\langle \text{result} \rangle$ does not influence these values. $\langle \text{distrib} \rangle$ can only be taken if $v_j(a) \neq 0$ and we thus have $J(a, n+1) = \emptyset$. Importantly, the condition is in fact even preserved by snipes, since properties of the form “For all agents $\dots$ ” become easier by removing agents. Additionally note that the condition holds in any initial configuration.

Next we prove that cases 1 and 2 are non-problematic. We first claim that ρ ends in a terminal configuration.

Proof of claim: wlog we reorder ρ s.t. ρ is of the form $C_0 \rightarrow^* C \hookrightarrow^* D \rightarrow^* C_{term}$, i.e. such that all snipes happen at once and it reaches a terminal configuration. To ensure this, we commute the snipes past other transitions towards the last snipe. This is possible since a non-interacting agent might as well not exist. Transitions **<steal>** and **<retire>** reduce the total number of non-zero entries in the copies of $\mathbb{Z}_m$ and this number cannot be increased. Hence they become permanently disabled eventually. Then all non-leaders have all entries equal to 0, and **<distrib>** is hence also disabled. Now **<result>** provides every agent with some output for every subprotocol, and afterwards becomes disabled.

Hence ρ is of the form $C_0 \rightarrow^* C \hookrightarrow^* D \rightarrow^* C_{term}$. Let j be one of the copies such that no leader for this copy was sniped (i.e. such that no case 3 occurred for this copy). We prove that the projection pr_j of ρ to the j -th subprotocol has the correct output. For all agents which were sniped before ever interacting, we assume they were sniped at the start, and hence consider the initial configuration C'_0 with those agents removed. C'_0 still has at least $2m$ agents. Consider the execution ρ' starting at C'_0 and otherwise mirroring $\text{pr}_j(\rho)$, which is possible since the removed agents never interacted in the j -th subprotocol. We prove that the j -th subprotocol has the correct output in ρ' , and hence also in ρ .

By choice of j , the only agents which are sniped in ρ' fulfilled $v(j) = 0$, and were non-leaders for this copy. In particular after the first **<distrib>**, every configuration in ρ' always had at least one leader, in particular the terminal configuration $\text{pr}(C_{term})$. Since the configuration is terminal, we have exactly one leader. The j -th sum $\text{Sum}_j(C) := \sum_{q=(i,v,r) \in Q} v(j) \cdot C(q)$ is invariant modulo m , and is not impacted by sniping of agents with $v(j) = 0$. Hence we in particular have $\text{Sum}_j(\text{pr}(C_{term})) \equiv \text{Sum}_j(C'_0) \pmod{m}$. In $\text{pr}(C_{term})$ this value is fully stored by the leader of the j -th subprotocol, since otherwise **<steal>** would be enabled. Hence the leader has the correct output, and therefore using transition **<result>** every agent eventually has the correct output in the j -th subprotocol.

We have therefore seen that only copies j where a leader gets sniped are damaged. Since every agent is leader in at most one subprotocol, and at most $\text{InTol}(C_0)$ many agents are sniped, it follows that at most $k \leq m - 1$ subprotocols have the wrong output. The majority $m + 1$ of subprotocols hence remain correct. Therefore the whole protocol has the correct output. $\square$

5.2 Handling Small Inputs

Sensei is not fully satisfied with her modulo protocol. At the moment the ninjas have a lot of members (thanks to the many new rookies), but that can change at any moment. Therefore it is important for the modulo protocol to also handle small inputs. She thinks of the following idea: a tower computes the predicate $\psi' := \sum_{i=1}^n x_i \geq 2m$ to detect if there are enough ninjas to successfully use the BIGMODULO protocol. If there are not enough ninjas, she would want to determine the exact modulo value by the highest ninja in the tower, which is not correct with this choice of ψ' , since the coefficients were ignored. Hence instead she uses $\psi := \sum_{i=1}^n a_i x_i \geq 3m^2$, which is a sufficient condition that there

are at least $3m$ ninjas, though not a necessary condition.

Since the ninjas do not know how many of them will show up, in particular they do not know whether their number will be large or small. We solve this problem by suitably combining the protocols for the two cases. Intuitively, the ninjas execute both protocols *simultaneously*, that is, at every moment in time they are in a state for each protocol. Further, each ninja maintains an estimate of their remainder modulo m . Formally, we let $\mathcal{P}_T = (Q_T, \delta_T, _, O_T) = \text{INHOMTOWER}$ for the predicate ψ and $\mathcal{P}_M = (Q_M, \delta_M, _, O_M) = \text{BIGMODULO}$. The **MODULOCOMBINED** population protocol is defined as follows:

Protocol MODULOCOMBINED.

States. $Q = Q_T \times [0, 3m^2] \times Q_M$
Input. $I = \{([0, a_i], a_i, (0, a_i \cdot \mathbb{1}_{[1, 2m]}, 0)) : 1 \leq i \leq n\}$
Output. $O((q_T, h, q_M)) = \begin{cases} h \bmod m \geq t & \text{if } h < 3m^2 \\ O_M(q_M) & \text{else} \end{cases}$
Transitions.

- For all $(q_1, q_2 \mapsto q'_1, q'_2) \in \delta_T$ and $(p_1, p_2 \mapsto p'_1, p'_2) \in \delta_M$ and $h_1, h_2 \in [0, 3m^2]$ we add the following transition:

$$(q_1, h_1, p_1), (q_2, h_2, p_2) \mapsto (q'_1, h, p'_1), (q'_2, h, p'_2) \quad \langle \text{parallel} \rangle$$

Where $h = \max\{h_1, h_2, \text{END}(q'_1), \text{END}(q'_2)\}$ with $\text{END}([s, e)) = e$.

Theorem 4. *MODULOCOMBINED computes φ and is robust.*

Proof. Let C_0 be an initial configuration, and $\rho = (C_0, C_1, \dots)$ a fair k -execution with $k = \text{InTol}(C_0)$. Since every fair execution of both **BIGMODULO** and **INHOMTOWER** terminates, every fair execution of **MODULOCOMBINED** terminates. Let C_r be the terminal configuration reached. Since $\langle \text{parallel} \rangle$ in particular updates the values of h_1 and h_2 to the maximum, the fact that C_r is terminal implies that every agent has the same value h in this component. Since we chose the threshold of ψ to be $3m^2$, which is a multiple of m , we have that $\text{InTol}_{\mathcal{P}_T}(C_0) \geq \text{InTol}_{\mathcal{P}_M}(C_0)$. By this argument the following case distinction does not depend on snipes:

Case 1: $h < 3m^2$. Then no agent in the tower component is at the top, and the output of **MODULOCOMBINED** is determined by O_T . Let C'_0 be the initial configuration if the agents sniped in ρ had been sniped at the start. The height h has to be in the interval $[\sum_{q \in Q} \text{len}(q_T) \cdot C'_0(q), \min(\sum_{q \in Q} \text{len}(q_T) \cdot C_0(q), 3m^2)]$. By definition of initial tolerance, in fact φ is constant even on the larger interval $[\sum_{q \in Q} \text{len}(q_T) \cdot C'_0(q), \sum_{q \in Q} \text{len}(q_T) \cdot C_0(q)]$. Hence no matter which of these values h actually has, using it we obtain the correct value of φ .

Case 2: $h = 3m^2$. Then the output is fully given by $O_M(q_M)$. Since $h = 3m^2$, and every coefficient a_i is at most m , initially we must have had at least $3m$ agents. Hence C_0 is a large enough initial configuration that we can use **Proposition 5** which states **BIGMODULO** is robust for C_0 , wherefore the projection of ρ to Q_M and therefore also ρ itself gives the correct output. $\square$

6 Future Work: Boolean Combinations

Recall that every Presburger predicate can be represented as a boolean combination of threshold and modulo predicates. Given protocols for two predicates φ_1, φ_2 , there is a simple construction that yields protocols deciding $\varphi_1 \wedge \varphi_2$ or $\varphi_1 \vee \varphi_2$. Intuitively, agents execute both protocols simultaneously. In particular, their states are pairs of states of the protocols for φ_1 and φ_2 , and their output function is the conjunction of disjunction of the output functions for φ_1 and φ_2 [3]. If this construction would preserve robustness, that is, if the protocol for $\varphi_1 \wedge \varphi_2$ or $\varphi_1 \vee \varphi_2$ would be robust whenever the protocols for φ_1 and φ_2 are, then we would have proved that every predicate has a robust protocol.

Unfortunately, the construction does not preserve robustness. While applying the construction for conjunction to our `INHOMTOWER` protocol from [Section 4.1](#) yields a robust protocol, this is not the case for the `INHOMTOWERCANCEL` protocol from [Section 4.3](#).

Consider an initial configuration where the first protocol accepts and the second protocol rejects. Additionally, via sniping within the initial tolerance, it is possible to make the first protocol reject and the second accept. Now the sniper can wait until the first protocol accepts and then snipe such that the second protocol also accepts.

Example 3. We consider the two predicates $\varphi_1(x, y) = (2x - y \geq 3)$ and $\varphi_2(x, y) = (y - x \geq 1)$ and their conjunction $\varphi(x, y) = (\varphi_1(x, y) \wedge \varphi_2(x, y))$. The reachable states of our `INHOMTOWERCANCEL` protocol instantiated with the two predicates are:

$$Q_1 = \{[0, 2), [1, 3), -1, 0\} \quad Q_2 = \{[0, 1), -1, 0\}$$

Where the initial states corresponding to x are $[0, 2)$ for φ_1 and -1 for φ_2 . For y we have -1 and $[0, 1)$ respectively.

In order to show that the construction is not robust, we search for an initial configuration that is rejected by φ_1 and accepted by φ_2 , and which can be sniped such that φ_1 accepts and φ_2 rejects. From such a configuration we can have an execution that first stabilizes in the second protocol, then a snipe follows, and then the first protocol also stabilizes, leading to an overall accepting configuration that should have been rejected. To find such a configuration, we draw the plot in [Figure 4](#). Colors indicate the values of both predicates (see the legend of the figure). Sniping corresponds to moving left or down in the plot, and so we search for a blue square from which we can reach a red square by moving down or left, and from which we cannot reach a black square with the same number of moves. This last condition corresponds to the tolerance condition: the sniper is only allowed to snipe when the output of the initial configuration does not change.

The plot shows that $C = \{3 \cdot x, 4 \cdot y\}$ is a suitable configuration. Its initial tolerance is 6, since all configurations $D \leq C$ are rejected. We now consider the following fair

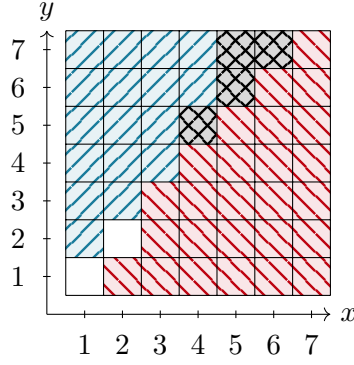


Figure 4: Plot of the predicates $\varphi_1(x, y) = (2x - y \geq 3)$ and $\varphi_2(x, y) = (y - x \geq 1)$ on initial configurations. The color indicates the value of the predicates: it is red if φ_1 accepts, blue if φ_2 accepts, black if both accept and white if both reject.

1-execution (state changes are highlighted in bold):

$$\{3 \cdot ([0, 2], -1), 4 \cdot (-1, [0, 1])\} \quad (1)$$

$$\rightarrow^* \{3 \cdot ([0, 2], \mathbf{0}), 3 \cdot (-1, \mathbf{0}), \mathbf{1} \cdot (-1, [0, 1])\} \quad (2)$$

$$\hookrightarrow \{3 \cdot ([0, 2], 0), \mathbf{2} \cdot (-1, 0), (-1, [0, 1])\} \quad (3)$$

$$\rightarrow^* \{\mathbf{1} \cdot ([0, 2], 0), 2 \cdot ([\mathbf{1}, \mathbf{3}], 0), 2 \cdot (-1, 0), (-1, [0, 1])\} \quad (4)$$

$$\rightarrow^* \{([0, 2], 0), 2 \cdot ([1, \mathbf{2}], 0), 2 \cdot (\mathbf{0}, 0), (-1, [0, 1])\} \quad (5)$$

$$\rightarrow^* \{([0, 2], 0), 2 \cdot ([\mathbf{2}, \mathbf{3}], 0), 2 \cdot (0, 0), (-1, [0, 1])\} \quad (6)$$

$$\rightarrow^* \{([0, 2], 0), \mathbf{1} \cdot ([2, 3], 0), \mathbf{3} \cdot (0, 0), (\mathbf{0}, [0, 1])\} \quad (7)$$

From (1) to (2) only the second protocol executes a non-silent transition: three of the $(-1, [0, 1])$ agents annihilate with $([0, 2], -1)$ in the second protocol. Then in (3), we proceed to snipe a single $(-1, 0)$ agent. This does not influence the second protocol, since it already is finished. In (4), the $([0, 2], 0)$ agents move upwards in the first protocol and after that in (5), we annihilate $([2, 3], 0)$ with $(-1, 0)$ twice. In (6) we again move the agents in the first protocol upwards and in (7) we do a final annihilation.

The last configuration is accepting and terminal, thus we have shown that the product construction applied to our protocol is not robust.

This example only shows that the simple construction for conjunction does not preserve robustness. But there could be a more sophisticated construction, or a procedure that directly constructs a robust protocol for any predicate, for example from a finite automaton for the predicate. Currently, the simplest predicate for which we have not been able to find a robust protocol is

$$\varphi(x, y) = (x \geq y \vee (x + 1 \geq y \wedge x \bmod 5 = 0))$$

The problematic initial configurations are of the form $\{5k \cdot x, n \cdot y\}$ with $5k > n$. The initial tolerance of these configurations is $5k - n$, but the robustness of the majority protocol only gives correct results for up to $5k - n - 1$ snipes.

References

- [1] Alistarh, D., Dudek, B., Kosowski, A., Soloveichik, D., Uznanski, P.: Robust detection in leak-prone population protocols. CoRR **abs/1706.09937** (2017), <http://arxiv.org/abs/1706.09937>
- [2] Alistarh, D., Töpfer, M., Uznanski, P.: Comparison dynamics in population protocols. In: Miller, A., Censor-Hillel, K., Korhonen, J.H. (eds.) PODC '21: ACM Symposium on Principles of Distributed Computing, Virtual Event, Italy, July 26-30, 2021. pp. 55–65. ACM (2021). <https://doi.org/10.1145/3465084.3467915>, <https://doi.org/10.1145/3465084.3467915>
- [3] Angluin, D., Aspnes, J., Diamadi, Z., Fischer, M.J., Peralta, R.: Computation in networks of passively mobile finite-state sensors. *Distributed Comput.* **18**(4), 235–253 (2006). <https://doi.org/10.1007/S00446-005-0138-3>, <https://doi.org/10.1007/s00446-005-0138-3>
- [4] Angluin, D., Aspnes, J., Eisenstat, D., Ruppert, E.: The computational power of population protocols. *Distributed Comput.* **20**(4), 279–304 (2007)
- [5] Blondin, M., Esparza, J., Jaax, S., Kucera, A.: Black ninjas in the dark: Formal analysis of population protocols. In: LICS. pp. 1–10. ACM (2018)
- [6] Czerner, P., Guttenberg, R., Helfrich, M., Esparza, J.: Fast and succinct population protocols for presburger arithmetic. *J. Comput. Syst. Sci.* **140**, 103481 (2024)
- [7] Delporte-Gallet, C., Fauconnier, H., Guerraoui, R., Ruppert, E.: When birds die: Making population protocols fault-tolerant. In: Gibbons, P.B., Abdelzaher, T.F., Aspnes, J., Rao, R.R. (eds.) *Distributed Computing in Sensor Systems*, Second IEEE International Conference, DCOSS 2006, San Francisco, CA, USA, June 18-20, 2006, Proceedings. *Lecture Notes in Computer Science*, vol. 4026, pp. 51–66. Springer (2006). https://doi.org/10.1007/11776178_4, https://doi.org/10.1007/11776178_4
- [8] Guerraoui, R., Ruppert, E.: Names trump malice: Tiny mobile agents can tolerate byzantine failures. In: Albers, S., Marchetti-Spaccamela, A., Matias, Y., Nikolettseas, S.E., Thomas, W. (eds.) *Automata, Languages and Programming*, 36th International Colloquium, ICALP 2009, Rhodes, Greece, July 5-12, 2009, Proceedings, Part II. *Lecture Notes in Computer Science*, vol. 5556, pp. 484–495. Springer (2009). https://doi.org/10.1007/978-3-642-02930-1_40, https://doi.org/10.1007/978-3-642-02930-1_40